

Lab Assignment 3

CMPE 146 Section 1

Professor Bhawandeep Singh Harsh

Andrew Bertch

Robert Trinh

Table of Contents

Exercise 1 – Time Measurement.....	3
Exercise 2 – CRC Accelerator	4
Exercise 2.1 – Simple Checksum.....	4
Exercise 2.2 – Speedup	5
Exercise 2.3 – Simulate Data Corruption.....	7
Exercise 3 – Interrupt.....	10
Exercise 3.1 – Control	10
Exercise 3.2 - Measurement.....	12
Appendix A – MCU Source Code.....	13
simple_measure.c	13
simple_checksum.c	15
speedup_checksum.c	17
corrupt_data.c.....	22
LED_interrupt.c.....	27
time_interrupt.c.....	30

Exercise 1 – Time Measurement

In this exercise, we create a program that measures how long the button, S2, is pressed for by using a 32-bit counter. Some of the outputs of the program are included below:

CORTEX_MoP: Duration: 192.287 ms

CORTEX_MoP: Duration: 100.680 ms

CORTEX_MoP: Duration: 378.753 ms

CORTEX_MoP: Duration: 2034.191 ms

CORTEX_MoP: Duration: 1105.693 ms

CORTEX_MoP: Duration: 647.568 ms

The relevant code for measuring the time the button has been held down is shown below:

```
else if (status_pb != 0 && released == 0) {
    released = 1;
    released_value = DL_Timer_getTimerCount(TIMG12);
    double ms = ((double)released_value - pressed_value) /
(32000000) * 1000;
    printf("Duration: %.3f ms\n", ms);
}
```

Given the operating parameters of the LaunchPad MCU, it would take the counter of 0xFFFFFFFF 134 seconds to rollover. This can be found by taking the max size of an unsigned 32-bit number, 2^{32} , and dividing it by the clock frequency, 32 Mhz.

$$\frac{2^{32}}{32,000,000} = 134 \text{ seconds}$$

Exercise 2 – CRC Accelerator

In this exercise, we explore different methods to compute a 32-bit checksum of a block of data. We'll make use of the CRC accelerator on the MCU to generate the CRC checksum and also use an algorithm to create a checksum using software. The speeds and effectiveness of the different checksum methods will be compared.

Exercise 2.1 – Simple Checksum

In this exercise, we will create a simple checksum that is generated by summing all the input data with the result being the checksum. To balance the bytes that get written to instead of writing to the least significant bit every time, we write our summing function to add the first byte to byte 0 of checksum, second byte to byte1, and so on. The summing function is shown below.

```
uint32_t compute_simple_checksum(uint8_t* data, uint32_t length)
{
    uint32_t checksum = 0;

    for(int i = 0; i < length; i++)
    {
        if(i % 4 == 0)
            checksum += (uint32_t)data[i];
        else if(i % 4 == 1)
            checksum += (uint32_t)(data[i] << 8);
        else if(i % 4 == 2)
            checksum += (uint32_t)(data[i] << 16);
        else if(i % 4 == 3)
            checksum += (uint32_t)(data[i] << 24);
    }

    checksum = ~(checksum);
}
```

```

    return checksum;
}

```

Testing the function using the same data array size and the same seed for the random number generator results in the outputs shown below.

CORTEX_MoP: First four bytes: DC, 04, 65, AA

CORTEX_MoP: Last four bytes: B2, 8C, AF, 7C

CORTEX_MoP: Checksum: FFDDE233

A potential change to the algorithm we implemented to speed up computation could be to remove the if statement checking which byte to add the data to from the loop. Each byte in the 32-bit checksum can be addressed using a pointer. This would remove the if statement which adds instructions and cycles to check for branching and the logical shifts that occur to add the correct data value to

Exercise 2.2 – Speedup

In this exercise, we set up two different ways to compute the CRC checksum, hardware and software. The two methods are compared with the speed of the simple checksum function also being recorded. The code that does the measurements of the speed of each checksum generation method is shown below.

```

start_value_one = DL_Timer_getTimerCount(TIMG12);
for(int i = 0; i < 10240; i++)
{
    DL_CRC_feedData8(CRC, myData[i]);
}
uint32_t checksum_one = DL_CRC_getResult32(CRC);
end_value_one = DL_Timer_getTimerCount(TIMG12);
time_one = ((double)end_value_one - start_value_one) / (32000000)
* 1000000;

```

```

start_value_two = DL_Timer_getTimerCount(TIMG12);
uint32_t checksum_two = calculateCRC32(myData, 10240);
end_value_two = DL_Timer_getTimerCount(TIMG12);
time_two = ((double)end_value_two - start_value_two) / (32000000)
* 1000000;

```

```

start_value_three = DL_Timer_getTimerCount(TIMG12);
uint32_t checksum_three = compute_simple_checksum(myData, 10240);
end_value_three = DL_Timer_getTimerCount(TIMG12);
time_three = ((double)end_value_three - start_value_three) /
(32000000) * 1000000;

```

The outputs on the console of the program are shown below. Included are the bytes of the simple checksum function, the measured times of all checksum methods, the generated checksums, and the speed up of the CRC accelerometer method versus the software CRC method.

CORTEX_MoP: First four bytes: DC, 04, 65, AA

CORTEX_MoP: Last four bytes: B2, 8C, AF, 7C

CORTEX_MoP: Time one: 1448.563 us, Checksum one: 9991F447

CORTEX_MoP: Time two: 22490.406 us, Checksum two: 9991F447

CORTEX_MoP: Time three: 1320.438 us, Checksum three: FFDDE233

CORTEX_MoP: Ratio of speedup: 15.526

Looking at the measured times and the resulting speed up, using the dedicated CRC hardware is much faster to generate a CRC checksum. This is most likely because the

use of dedicated hardware will make the calculation much faster since the CRC accelerator is optimized to quickly calculate the CRC value. It can take advantage of digital circuits that are optimized for this calculation as opposed to the general-purpose circuits and instructions available to the software CRC method. While the internal of how the accelerator functions with the DriverLib library abstracting it, it would most likely be quicker due to the pure software and CPU instruction method using nested for loops. This would be faster since loops add significant cycles to compute time and nesting them compounds the cycles added.

Exercise 2.3 – Simulate Data Corruption

In this exercise, we used the DriverLib CRC checksum method and the simple checksum method and simulated data corruption in calculating the checksums. The code that does this is shown below.

```
myData[2496] ^= 0x01;

// CRC setup
DL_CRC_reset(CRC);
DL_CRC_enablePower(CRC);
delay_cycles(POWER_STARTUP_DELAY);
DL_CRC_init(CRC, DL_CRC_32_POLYNOMIAL, DL_CRC_BIT_REVERSED,
DL_CRC_INPUT_ENDIANESS_LITTLE_ENDIAN,
DL_CRC_OUTPUT_BYTESWAP_DISABLED);
DL_CRC_setSeed32(CRC, CRC32_SEED);
for(int i = 0; i < 10240; i++)
{
    DL_CRC_feedData8(CRC, myData[i]);
}
uint32_t checksum_two_hardware = DL_CRC_getResult32(CRC);
uint32_t checksum_two_def = compute_simple_checksum(myData,
10240);
```

```

myData[8864] ^= 0x01;

// CRC setup
DL_CRC_reset(CRC);
DL_CRC_enablePower(CRC);
delay_cycles(POWER_STARTUP_DELAY);
DL_CRC_init(CRC, DL_CRC_32_POLYNOMIAL, DL_CRC_BIT_REVERSED,
DL_CRC_INPUT_ENDIANESS_LITTLE_ENDIAN,
DL_CRC_OUTPUT_BYTESWAP_DISABLED);
DL_CRC_setSeed32(CRC, CRC32_SEED);
for(int i = 0; i < 10240; i++)
{
    DL_CRC_feedData8(CRC, myData[i]);
}

uint32_t checksum_three_hardware = DL_CRC_getResult32(CRC);
uint32_t checksum_three_def = compute_simple_checksum(myData,
10240);

```

The console outputs for all 3 calculated checksums for each method is shown below.

CORTEX_MoP: Checksum one hardware: 9991F447

CORTEX_MoP: Checksum one def: FFDDE233

CORTEX_MoP: Checksum two hardware: 48CF5BoF

CORTEX_MoP: Checksum two def: FFDDE232

CORTEX_MoP: Checksum three hardware: 06B66622

CORTEX_MoP: Checksum three def: FFDDE233

Between the two methods, we can see that the hardware method generates a different checksum for each set of data. The corrupted values generate a different checksum. The simple checksum generator however generates the same checksum for all 3 data sets with the corrupted values generating the same checksum as the non-corrupted data set.

Based on these results, the hardware CRC checksum method is a better checksum method since it generates different values for single bit corruptions in the data it is generating a checksum value for. This is very useful since it can accurately tell us if we are getting the correct data that we expect.

Exercise 3 – Interrupt

In this exercise, we will make use of interrupts to provide quick responses to external events. Using interrupts allows us to respond quicker to events, such as button presses, and instead of having our control loop keep checking the device occasionally we can instead have the device generate an interrupt when it changes. This whole exercise will include us controlling an LED and then measuring the time the button has been pressed.

Exercise 3.1 – Control

In this part of the exercise, we will use an interrupt to control an LED. This is almost identical to what we had in Lab 2 Exercise 3. The ISR that achieves this function is shown below.

```
void GROUP1_IRQHandler(void)
{
    // Get the pending interrupt
    uint32_t interrupt_status =
DL_GPIO_getEnabledInterruptStatus(GPIOB, DL_GPIO_PIN_21);
    if ((interrupt_status & DL_GPIO_PIN_21) == DL_GPIO_PIN_21) // From
S2
    {
        // Handle the interrupt here

        if (released == 1) {
            released = 0;
            DL_GPIO_setPins(GPIOB, DL_GPIO_PIN_22);

        }
        else if (released == 0) {
            released = 1;
            DL_GPIO_clearPins(GPIOB, DL_GPIO_PIN_22);
        }
    }
}
```

```

    }

    DL_GPIO_clearInterruptStatus(GPIOB, DL_GPIO_PIN_21);
}
}

```

The control is the same as that from Lab 2 Exercise 3. The control logic checks the status of the button and if the button has been released or not. If the button has been pressed and the button has been released before (starts with released set to 1) then the state is inverted. If the button isn't being pressed and the button was previously pressed (released = 0) then released is reset to 1. At the end, the LED is set to state.

Compared with the polling method from Lab 2, this method does not currently provide a quicker response. This is because the control loop from Lab 2 Exercise 3 only had the LED polling in it. This meant that the control loop was constantly checking for the button to be pressed, similar to what an interrupt would achieve when invoking an ISR. The interrupt method would be more responsive in a more complex control loop with more occurring since it could immediately have the MCU complete the interrupt instead of waiting for the control loop to loop back around to the polling point.

Exercise 3.2 - Measurement

In this exercise, we add time measurement to the interrupt previously implemented. The interrupt only records when the button has been pressed or released with the main control loop handling the calculations. This is to keep the interrupt simple and keep more complex tasks out of the interrupt. Keeping it out of the interrupt ensures that the ISR is quick and does not take too long. To facilitate the communication between the ISR and main() we make use of a circular queue. A sample of console outputs is shown below which also shows the circular queue functioning.

CORTEX_MoP: Duration: 111.402 ms, Read Index: 1

CORTEX_MoP: Duration: 600.345 ms, Read Index: 3

CORTEX_MoP: Duration: 61.157 ms, Read Index: 5

CORTEX_MoP: Duration: 639.403 ms, Read Index: 7

CORTEX_MoP: Duration: 1369.765 ms, Read Index: 9

CORTEX_MoP: Duration: 604.703 ms, Read Index: 1

CORTEX_MoP: Duration: 246.817 ms, Read Index: 3

CORTEX_MoP: Duration: 63.629 ms, Read Index: 5

CORTEX_MoP: Duration: 288.447 ms, Read Index: 7

The array queue being only 10 items long should be enough for our use case since the device that we are checking doesn't change that rapidly. It is unlikely that it would be enough for the interrupt to fill the queue before the main loop can process all of the data.

Appendix A – MCU Source Code

simple_measure.c

```
#include "ti_msp_dl_config.h"

#include <stdio.h>

int main(void)
{
    SYSCFG_DL_init();

    DL_GPIO_enablePower(GPIOB);

    DL_GPIO_initPeripheralInputFunctionFeatures(IOMUX_PINCM49,
    IOMUX_PINCM49_PF_GPIOB_DIO21, DL_GPIO_INVERSION_DISABLE,
    DL_GPIO_RESISTOR_PULL_UP, DL_GPIO_HYSTERESIS_DISABLE,
    DL_GPIO_WAKEUP_DISABLE);

    // Set up to use the 32-bit counter TIMG12
    DL_Timer_enablePower(TIMG12);

    DL_Timer_ClockConfig config;
    config.clockSel = DL_TIMER_CLOCK_BUSCLK;
    config.divideRatio = DL_TIMER_CLOCK_DIVIDE_1;
    config.prescale = 0;
    DL_Timer_setClockConfig(TIMG12, &config);

    DL_Timer_TimerConfig timerConfig;
    timerConfig.timerMode = DL_TIMER_TIMER_MODE_PERIODIC_UP;
    timerConfig.period = -1;
    timerConfig.startTimer = DL_TIMER_START;
    timerConfig.genIntermInt = DL_TIMER_INTERM_INT_DISABLED;
```

```

timerConfig.counterVal = 0;
DL_Timer_initTimerMode(TIMG12, &timerConfig);


uint32_t status_pb;
uint32_t pressed_value;
uint32_t released_value;
int released = 1;


while (1) {

    status_pb = DL_GPIO_readPins(GPIOB, DL_GPIO_PIN_21);

    if (status_pb == 0 && released == 1) {
        released = 0;
        pressed_value = DL_Timer_getTimerCount(TIMG12);

    }
    else if (status_pb != 0 && released == 0) {
        released = 1;
        released_value = DL_Timer_getTimerCount(TIMG12);
        double ms = ((double)released_value - pressed_value) /
(32000000) * 1000;
        printf("Duration: %.3f ms\n", ms);
    }
}
}

```

simple_checksum.c

```
#include "ti_msp_dl_config.h"
#include <stdio.h>
#include <stdlib.h>

uint32_t compute_simple_checksum(uint8_t* data, uint32_t length)
{
    uint32_t checksum = 0;

    for(int i = 0; i < length; i++)
    {
        if(i % 4 == 0)
            checksum += (uint32_t)data[i];
        else if(i % 4 == 1)
            checksum += (uint32_t)(data[i] << 8);
        else if(i % 4 == 2)
            checksum += (uint32_t)(data[i] << 16);
        else if(i % 4 == 3)
            checksum += (uint32_t)(data[i] << 24);

    }

    checksum = ~(checksum);

    return checksum;
}
```

```
int main(void)
{
    SYSCFG_DL_init();
    static uint8_t myData[10240];

    srand(12345);
    for(int i = 0; i < 10240; i++)
    {
        myData[i] = rand();
    }

    printf("First four bytes: %02X, %02X, %02X, %02X \n", myData[0],
myData[1], myData[2], myData[3]);

    printf("Last four bytes: %02X, %02X, %02X, %02X \n",
myData[10236], myData[10237], myData[10238], myData[10239]);

    uint32_t checksum = compute_simple_checksum(myData, 10240);

    printf("Checksum: %08X\n", checksum);

    while (1) {

    }
}
```


speedup_checksum.c

```
#include "ti_msp_dl_config.h"
#include <stdio.h>
#include <stdlib.h>
#define CRC32_SEED 0xFFFFFFFF

/* Standard software calculation of CRC32 */
static uint32_t calculateCRC32(uint8_t* data, uint32_t length)
{
    const uint32_t crc32_poly = 0xEDB88320;
    uint32_t ii, jj, byte, crc, mask;
    crc = CRC32_SEED;
    for(ii=0;ii<length;ii++)
    {
        byte = data[ii];
        crc = crc ^ byte;
        for (jj = 0; jj < 8; jj++)
        {
            mask = -(crc & 1);
            crc = (crc >> 1) ^ (crc32_poly & mask);
        }
    }
    return crc;
}

uint32_t compute_simple_checksum(uint8_t* data, uint32_t length)
{

```

```

uint32_t checksum = 0;

for(int i = 0; i < length; i++)
{
    if(i % 4 == 0)
        checksum += (uint32_t)data[i];
    else if(i % 4 == 1)
        checksum += (uint32_t)(data[i] << 8);
    else if(i % 4 == 2)
        checksum += (uint32_t)(data[i] << 16);
    else if(i % 4 == 3)
        checksum += (uint32_t)(data[i] << 24);
}

checksum = ~(checksum);

return checksum;
}

int main(void)
{
    SYSCFG_DL_init();

    DL_GPIO_enablePower(GPIOB);

    DL_GPIO_initPeripheralInputFunctionFeatures(IOMUX_PINCM49,
IOMUX_PINCM49_PF_GPIOB_DIO21, DL_GPIO_INVERSION_DISABLE,

```

```

DL_GPIO_RESISTOR_PULL_UP, DL_GPIO_HYSTERESIS_DISABLE,
DL_GPIO_WAKEUP_DISABLE);

    // Set up to use the 32-bit counter TIMG12

    DL_Timer_enablePower(TIMG12);

    DL_Timer_ClockConfig config;
    config.clockSel = DL_TIMER_CLOCK_BUSCLK;
    config.divideRatio = DL_TIMER_CLOCK_DIVIDE_1;
    config.prescale = 0;
    DL_Timer_setClockConfig(TIMG12, &config);

    DL_Timer_TimerConfig timerConfig;
    timerConfig.timerMode = DL_TIMER_TIMER_MODE_PERIODIC_UP;
    timerConfig.period = -1;
    timerConfig.startTimer = DL_TIMER_START;
    timerConfig.genIntermInt = DL_TIMER_INTERM_INT_DISABLED;
    timerConfig.counterVal = 0;
    DL_Timer_initTimerMode(TIMG12, &timerConfig);


    // CRC setup
    DL_CRC_reset(CRC);
    DL_CRC_enablePower(CRC);
    delay_cycles(POWER_STARTUP_DELAY);
    DL_CRC_init(CRC, DL_CRC_32_POLYNOMIAL, DL_CRC_BIT_REVERSED,
    DL_CRC_INPUT_ENDIANESS_LITTLE_ENDIAN,
    DL_CRC_OUTPUT_BYTESWAP_DISABLED);
    DL_CRC_setSeed32(CRC, CRC32_SEED);


    uint32_t start_value_one, start_value_two, start_value_three;

```

```

uint32_t end_value_one, end_value_two, end_value_three;
double time_one, time_two, time_three;

static uint8_t myData[10240];

srand(12345);
for(int i = 0; i < 10240; i++)
{
    myData[i] = rand();
}

printf("First four bytes: %02X, %02X, %02X, %02X\n", myData[0],
myData[1], myData[2], myData[3]);

printf("Last four bytes: %02X, %02X, %02X, %02X\n", myData[10236],
myData[10237], myData[10238], myData[10239]);

start_value_one = DL_Timer_getTimerCount(TIMG12);
for(int i = 0; i < 10240; i++)
{
    DL_CRC_feedData8(CRC, myData[i]);
}
uint32_t checksum_one = DL_CRC_getResult32(CRC);
end_value_one = DL_Timer_getTimerCount(TIMG12);
time_one = ((double)end_value_one - start_value_one) / (32000000)
* 1000000;

start_value_two = DL_Timer_getTimerCount(TIMG12);
uint32_t checksum_two = calculateCRC32(myData, 10240);

```

```

    end_value_two = DL_Timer_getTimerCount(TIMG12);

    time_two = ((double)end_value_two - start_value_two) / (32000000)
* 1000000;

    start_value_three = DL_Timer_getTimerCount(TIMG12);
    uint32_t checksum_three = compute_simple_checksum(myData, 10240);
    end_value_three = DL_Timer_getTimerCount(TIMG12);
    time_three = ((double)end_value_three - start_value_three) /
(32000000) * 1000000;

    printf("Time one: %.3f us, Checksum one: %08X\n", time_one,
checksum_one);

    printf("Time two: %.3f us, Checksum two: %08X\n", time_two,
checksum_two);

    printf("Time three: %.3f us, Checksum three: %08X\n", time_three,
checksum_three);

    double ratio = time_two/time_one;
    printf("Ratio of speedup: %.3f\n", ratio);

    while (1) {

    }

}

```

corrupt_data.c

```
#include "ti_msp_dl_config.h"

#include <stdio.h>

#include <stdlib.h>

#define CRC32_SEED 0xFFFFFFFF

/* Standard software calculation of CRC32 */
static uint32_t calculateCRC32(uint8_t* data, uint32_t length)
{
    const uint32_t crc32_poly = 0xEDB88320;
    uint32_t ii, jj, byte, crc, mask;
    crc = CRC32_SEED;
    for(ii=0;ii<length;ii++)
    {
        byte = data[ii];
        crc = crc ^ byte;
        for (jj = 0; jj < 8; jj++)
        {
            mask = -(crc & 1);
            crc = (crc >> 1) ^ (crc32_poly & mask);
        }
    }
    return crc;
}

uint32_t compute_simple_checksum(uint8_t* data, uint32_t length)
{

```

```

uint32_t checksum = 0;

for(int i = 0; i < length; i++)
{
    if(i % 4 == 0)
        checksum += (uint32_t)data[i];
    else if(i % 4 == 1)
        checksum += (uint32_t)(data[i] << 8);
    else if(i % 4 == 2)
        checksum += (uint32_t)(data[i] << 16);
    else if(i % 4 == 3)
        checksum += (uint32_t)(data[i] << 24);

}

checksum = ~(checksum);

return checksum;
}

int main(void)
{
    SYSCFG_DL_init();

    static uint8_t myData[10240];

    srand(12345);

```

```

for(int i = 0; i < 10240; i++)
{
    myData[i] = rand();
}

// CRC setup
DL_CRC_reset(CRC);
DL_CRC_enablePower(CRC);
delay_cycles(POWER_STARTUP_DELAY);
DL_CRC_init(CRC, DL_CRC_32_POLYNOMIAL, DL_CRC_BIT_REVERSED,
DL_CRC_INPUT_ENDIANESS_LITTLE_ENDIAN,
DL_CRC_OUTPUT_BYTESWAP_DISABLED);
DL_CRC_setSeed32(CRC, CRC32_SEED);
for(int i = 0; i < 10240; i++)
{
    DL_CRC_feedData8(CRC, myData[i]);
}
uint32_t checksum_one_hardware = DL_CRC_getResult32(CRC);
uint32_t checksum_one_def = compute_simple_checksum(myData, 10240);

myData[2496] ^= 0x01;

// CRC setup
DL_CRC_reset(CRC);
DL_CRC_enablePower(CRC);
delay_cycles(POWER_STARTUP_DELAY);
DL_CRC_init(CRC, DL_CRC_32_POLYNOMIAL, DL_CRC_BIT_REVERSED,
DL_CRC_INPUT_ENDIANESS_LITTLE_ENDIAN,
DL_CRC_OUTPUT_BYTESWAP_DISABLED);

```



```

DL_CRC_setSeed32(CRC, CRC32_SEED);
for(int i = 0; i < 10240; i++)
{
    DL_CRC_feedData8(CRC, myData[i]);
}

uint32_t checksum_two_hardware = DL_CRC_getResult32(CRC);
uint32_t checksum_two_def = compute_simple_checksum(myData, 10240);


myData[8864] ^= 0x01;
// CRC setup
DL_CRC_reset(CRC);
DL_CRC_enablePower(CRC);
delay_cycles(POWER_STARTUP_DELAY);
DL_CRC_init(CRC, DL_CRC_32_POLYNOMIAL, DL_CRC_BIT_REVERSED,
DL_CRC_INPUT_ENDIANESS_LITTLE_ENDIAN,
DL_CRC_OUTPUT_BYTESWAP_DISABLED);
DL_CRC_setSeed32(CRC, CRC32_SEED);
for(int i = 0; i < 10240; i++)
{
    DL_CRC_feedData8(CRC, myData[i]);
}

uint32_t checksum_three_hardware = DL_CRC_getResult32(CRC);
uint32_t checksum_three_def = compute_simple_checksum(myData, 10240);


printf("Checksum one hardware: %08X\n", checksum_one_hardware);
printf("Checksum one def: %08X\n", checksum_one_def);
printf("Checksum two hardware: %08X\n", checksum_two_hardware);

```

```
printf("Checksum two def: %08X\n", checksum_two_def);  
printf("Checksum three hardware: %08X\n", checksum_three_hardware);  
printf("Checksum three def: %08X\n", checksum_three_def);
```

```
while (1) {
```

```
}
```

```
}
```

LED_interrupt.c

```
#include "ti_msp_dl_config.h"
```

```
#include <stdio.h>
```

```
int state = 0;
```

```
int released = 1;
```

```
void GROUP1_IRQHandler(void)
```

```
{
```

```
    // Get the pending interrupt
```

```
    uint32_t interrupt_status =
```

```
DL_GPIO_getEnabledInterruptStatus(GPIOB, DL_GPIO_PIN_21);
```

```
    if ((interrupt_status & DL_GPIO_PIN_21) == DL_GPIO_PIN_21) // From  
S2
```

```
    {
```

```
        // Handle the interrupt here
```

```
        uint8_t *BLED = &((uint8_t*)0x400A3214)[2];
```

```
        uint8_t *S2 = &((uint8_t*)0x400A3314)[1];
```

```
        if (*S2 == 0 && released == 1) {
```

```
            released = 0;
```

```
            state = !state;
```

```
        }
```

```
        else if (*S2 == 1 && released == 0) {
```

```
            released = 1;
```

```
        }
```

```
        *BLED = state;
```

```

        DL_GPIO_clearInterruptStatus(GPIOB, DL_GPIO_PIN_21);
    }
}

int main(void)
{
    SYSCFG_DL_init();

    DL_GPIO_enablePower(GPIOB);

    DL_GPIO_initDigitalOutput(IOMUX_PINCM57); // PB26, red LED in LED2
    DL_GPIO_initDigitalOutput(IOMUX_PINCM58); // PB27, green LED in
LED2
    DL_GPIO_initDigitalOutput(IOMUX_PINCM50); // PB22, blue LED in
LED2

    DL_GPIO_initDigitalInput(IOMUX_PINCM57); // PB26, red LED
    DL_GPIO_initDigitalInput(IOMUX_PINCM58); // PB27, green LED
    DL_GPIO_initDigitalInput(IOMUX_PINCM50); // PB22, blue LED
    DL_GPIO_initDigitalInput(IOMUX_PINCM49); // PB21, S2

    DL_GPIO_enableOutput(GPIOB, DL_GPIO_PIN_26); // PB26, red LED
    DL_GPIO_enableOutput(GPIOB, DL_GPIO_PIN_27); // PB27, green LED
    DL_GPIO_enableOutput(GPIOB, DL_GPIO_PIN_22); // PB22, blue LED

    DL_GPIO_clearPins(GPIOB, DL_GPIO_PIN_26);

```

```
DL_GPIO_clearPins(GPIOB, DL_GPIO_PIN_27);

DL_GPIO_clearPins(GPIOB, DL_GPIO_PIN_22);

DL_GPIO_clearPins(GPIOB, DL_GPIO_PIN_21);


DL_GPIO_initPeripheralInputFunctionFeatures(IOMUX_PINCM49,
IOMUX_PINCM49_PF_GPIOB_DIO21, DL_GPIO_INVERSION_DISABLE,
DL_GPIO_RESISTOR_PULL_UP, DL_GPIO_HYSTERESIS_DISABLE,
DL_GPIO_WAKEUP_DISABLE);


DL_GPIO_setUpperPinsPolarity(GPIOB,
DL_GPIO_PIN_21_EDGE_RISE_FALL);

DL_GPIO_clearInterruptStatus(GPIOB, DL_GPIO_PIN_21);

DL_GPIO_enableInterrupt(GPIOB, DL_GPIO_PIN_21);

NVIC_EnableIRQ(GPIOB_INT_IRQn);


while (1) {

}

}
```

time_interrupt.c

```
#include "ti_msp_dl_config.h"
```

```
#include <stdio.h>
```

```
int state = 0;
```

```
int released = 1;
```

```
#define QUEUE_SIZE 10
```

```
struct record {
```

```
uint32_t timestamp;
```

```
uint8_t state;
```

```
};
```

```
struct record queue[QUEUE_SIZE];
```

```
volatile int read_index;
```

```
volatile int write_index;
```

```
void GROUP1_IRQHandler(void)
```

```
{
```

```
    // Get the pending interrupt
```

```
    uint32_t interrupt_status =
```

```
DL_GPIO_getEnabledInterruptStatus(GPIOB, DL_GPIO_PIN_21);
```

```
    if ((interrupt_status & DL_GPIO_PIN_21) == DL_GPIO_PIN_21) // From  
S2
```

```
    {
```

```
        // Handle the interrupt here
```

```
        uint8_t *BLED = &((uint8_t*)0x400A3214)[2];
```

```
        uint8_t *S2 = &((uint8_t*)0x400A3314)[1];
```

```

    if (*S2 == 0 && released == 1) {
        released = 0;
        state = !state;
    }
    else if (*S2 == 1 && released == 0) {
        released = 1;
    }
    *BLED = state;

    int write_next = (write_index + 1) % QUEUE_SIZE;

    if(write_next == read_index)
    {
        printf("Cannot process new record\n");
        return;
    }

    uint32_t timestamp = DL_Timer_getTimerCount(TIMG12);
    uint8_t state = (DL_GPIO_readPins(GPIOB, DL_GPIO_PIN_21) !=
0);

    queue[write_index].timestamp = timestamp;
    queue[write_index].state = state;
    write_index++;
    if(write_index >= QUEUE_SIZE)
    {

```

```

        write_index = 0;
    }
    DL_GPIO_clearInterruptStatus(GPIOB, DL_GPIO_PIN_21);
}
}

int main(void)
{
    SYSCFG_DL_init();

    DL_GPIO_enablePower(GPIOB);

    DL_GPIO_initPeripheralInputFunctionFeatures(IOMUX_PINCM49,
    IOMUX_PINCM49_PF_GPIOB_DIO21, DL_GPIO_INVERSION_DISABLE,
    DL_GPIO_RESISTOR_PULL_UP, DL_GPIO_HYSTERESIS_DISABLE,
    DL_GPIO_WAKEUP_DISABLE);

    // Set up to use the 32-bit counter TIMG12
    DL_Timer_enablePower(TIMG12);
    DL_Timer_ClockConfig config;
    config.clockSel = DL_TIMER_CLOCK_BUSCLK;
    config.divideRatio = DL_TIMER_CLOCK_DIVIDE_1;
    config.prescale = 0;
    DL_Timer_setClockConfig(TIMG12, &config);
    DL_Timer_TimerConfig timerConfig;
    timerConfig.timerMode = DL_TIMER_TIMER_MODE_PERIODIC_UP;
    timerConfig.period = -1;
    timerConfig.startTimer = DL_TIMER_START;
    timerConfig.genIntermInt = DL_TIMER_INTERM_INT_DISABLED;

```



```
timerConfig.counterVal = 0;
```

```
DL_Timer_initTimerMode(TIMG12, &timerConfig);
```

```
DL_GPIO_enablePower(GPIOB);
```

```
DL_GPIO_initDigitalOutput(IOMUX_PINCM57); // PB26, red LED in LED2
```

```
DL_GPIO_initDigitalOutput(IOMUX_PINCM58); // PB27, green LED in  
LED2
```

```
DL_GPIO_initDigitalOutput(IOMUX_PINCM50); // PB22, blue LED in  
LED2
```

```
DL_GPIO_initDigitalInput(IOMUX_PINCM57); // PB26, red LED
```

```
DL_GPIO_initDigitalInput(IOMUX_PINCM58); // PB27, green LED
```

```
DL_GPIO_initDigitalInput(IOMUX_PINCM50); // PB22, blue LED
```

```
DL_GPIO_initDigitalInput(IOMUX_PINCM49); // PB21, S2
```

```
DL_GPIO_enableOutput(GPIOB, DL_GPIO_PIN_26); // PB26, red LED
```

```
DL_GPIO_enableOutput(GPIOB, DL_GPIO_PIN_27); // PB27, green LED
```

```
DL_GPIO_enableOutput(GPIOB, DL_GPIO_PIN_22); // PB22, blue LED
```

```
DL_GPIO_clearPins(GPIOB, DL_GPIO_PIN_26);
```

```
DL_GPIO_clearPins(GPIOB, DL_GPIO_PIN_27);
```

```
DL_GPIO_clearPins(GPIOB, DL_GPIO_PIN_22);
```

```
DL_GPIO_clearPins(GPIOB, DL_GPIO_PIN_21);
```

```
DL_GPIO_initPeripheralInputFunctionFeatures(IOMUX_PINCM49,  
IOMUX_PINCM49_PF_GPIOB_DIO21, DL_GPIO_INVERSION_DISABLE,  
DL_GPIO_RESISTOR_PULL_UP, DL_GPIO_HYSTERESIS_DISABLE,  
DL_GPIO_WAKEUP_DISABLE);
```

```
DL_GPIO_setUpperPinsPolarity(GPIOB,  
DL_GPIO_PIN_21_EDGE_RISE_FALL);
```

```
DL_GPIO_clearInterruptStatus(GPIOB, DL_GPIO_PIN_21);
```

```
DL_GPIO_enableInterrupt(GPIOB, DL_GPIO_PIN_21);
```

```
NVIC_EnableIRQ(GPIOB_INT_IRQn);
```

```
read_index = 0;
```

```
write_index = 0;
```

```
uint32_t pressed_value;
```

```
uint32_t released_value;
```

```
while (1) {
```

```
    delay_cycles(32000000 / 100); // Delay 10 ms at 32 MHz
```

```
    if(read_index != write_index)
```

```
    {
```

```
        struct record r = queue[read_index];
```

```
        if (r.state == 0) {
```

```
            pressed_value = r.timestamp;
```

```
        }
```

```
        else if (r.state == 1) {
            released_value = r.timestamp;
            double ms = ((double)released_value - pressed_value) /
(32000000) * 1000;
            printf("Duration: %.3f ms, Read Index: %d\n", ms,
read_index);
        }
        read_index++;
    }
    if(read_index >= QUEUE_SIZE)
    {
        read_index = 0;
    }
}
}
```